

Implementation and Evaluation of Latent Semantic Indexing (LSI) Search

Elyesa Duru

April 29, 2025

Contents

1	Introduction	2
2	What is Latent Semantic Indexing (LSI)?	2
3	Difference: With LSI vs Without LSI	2
4	How LSI Works: Simple Example	3
5	Code Snippets and Implementation	3
5.1	Creating the LSI Model	3
5.2	Performing a Search	3
5.3	Integrating into the GUI	4
6	Experiments and Examples	4
6.1	Observations	6
7	Conclusion	7
8	Purpose and Broader Context of This Information Retrieval (IR) System	7

1 Introduction

In this project, I extended my previous search engine by implementing **Latent Semantic Indexing (LSI)**. The goal was to allow the search system to retrieve documents not only based on exact keyword matches, but also based on the underlying semantic meaning of words. This is especially useful for improving search results when users use synonyms or related terms.

2 What is Latent Semantic Indexing (LSI)?

Latent Semantic Indexing (LSI) is a natural language processing technique that uncovers the hidden (“latent”) relationships between terms and documents. Instead of matching exact words, LSI maps documents and queries into a lower-dimensional “concept space” using a mathematical method called **Singular Value Decomposition (SVD)** [1].

The core idea behind LSI:

- Build a term-document matrix (words vs. documents). [1]
- Apply SVD to decompose this matrix into three smaller matrices. [1]
- Reduce the number of dimensions to capture only the most important “concepts”. [1]
- Represent both documents and queries in this compressed space. [1]
- Retrieve documents based on similarity in the concept space. [1]

Thus, LSI can find documents that are semantically similar, even if they do not share the exact same words.

3 Difference: With LSI vs Without LSI

In a traditional keyword search (like Boolean Search or TF-IDF Ranked Search):

- Documents are matched based on exact word occurrences.
- Synonyms or related concepts are not recognized.

With LSI:

- Documents are matched based on their *semantic similarity*.
- It captures hidden associations between words (e.g., “car” and “automobile”).
- It is more robust to synonyms and variations in language.

4 How LSI Works: Simple Example

Imagine you have the following documents:

- Document 1: “car engine wheels”
- Document 2: “racing car track”
- Document 3: “flowers garden plant”

In traditional keyword matching, searching for “automobile” would not return anything, since the word “automobile” does not appear.

With LSI:

- It identifies that “car”, “engine”, “racing” belong to a similar concept (vehicles).
- Even if “automobile” is used, it can retrieve documents about “cars” because they share the same latent semantic structure. [1].

5 Code Snippets and Implementation

The LSI model was integrated as follows: [2]

5.1 Creating the LSI Model

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
```

```
class LSISearch:
    def __init__(self, docs, n_components=100):
        """Initialize LSI model with documents."""
        self.docs = docs
        self.titles = list(docs.keys())
        self.n_components = n_components
        self._prepare_lsi()
```

5.2 Performing a Search

```
def search(self, query):
    """Performs LSI-based search."""
    query = preprocess(query)
    query_vec = self.vectorizer.transform([query])
    query_lsi = self.svd.transform(query_vec)

    # Compute cosine similarities
    similarities = np.dot(self.X_lsi, query_lsi.T).flatten()

    # Remove results where similarity is less than 0.0001 (close to 0)
    results = [(self.titles[idx], similarities[idx]) for idx in range(len(similarities) if similarities[idx] > 0.0001)]
```

```
# Rank documents by similarity (highest first)
results = sorted(results , key=lambda x: x[1] , reverse=True)

return results
```

5.3 Integrating into the GUI

The existing search interface was expanded by adding an “LSI Search” option to the dropdown menu.

```
search_type_menu = ttk.Combobox(root , textvariable=search_type_var , values=[
    "Boolean Search" ,
    "Boolean + Ranked Search" ,
    "LSI Search"
] , state="readonly")
```

During the search function, if “LSI Search” is selected, the ‘LSISearch.search()’ method is called instead of the standard Boolean or TF-IDF ranked search.

6 Experiments and Examples

Understanding the Role of `n_components` in LSI

In simple terms, the document collection is first represented as a high-dimensional matrix (terms $\times$ documents). Latent Semantic Indexing (LSI) uses Singular Value Decomposition (SVD) to reduce this matrix into a lower-dimensional space that captures the most important semantic relationships. The parameter `n_components` determines how many of those dimensions (or latent concepts) are retained — in other words, how many semantic topics we want the model to capture.

Why it’s important:

- A lower `n_components` value (e.g., 50) keeps only the most general topics but might lose important details.
- A higher value (e.g., 300 or more) captures more subtle nuances, but risks overfitting or including noise from less meaningful patterns.

Effect of `n_components` on Search Results

To better understand the influence of dimensionality on search quality, I experimented with the query "love" while varying the `n_components` parameter.

When setting `n_components = 100+`, the results from the LSI-based search closely match those from the ranked TF-IDF search — basically all of the documents returned contain the word "love" or revolve around romantic or emotional themes. This suggests that at this dimensionality, the model captures a detailed but still focused representation of term relationships.

However, when reducing the dimensionality to `n_components = 50 or less`, the results

become more diffuse. The model begins to include documents that are harder to associate directly with the theme of love, romance, or emotional depth. These documents sometimes do not even mention the keyword "love" at all, and their inclusion seems less intuitive.

Interestingly, with `n_components` set between 70 and 80, a few documents are retrieved that do not contain the term "love" but still match the overall emotional or romantic "vibe" of the query. This shows that LSI, at moderate dimensionality, is capable of identifying deeper, implicit connections between terms and document contexts — a major advantage over traditional keyword-based search.

Limitations Observed with Sparse Keywords

Further experiments using the keyword "car" revealed significantly worse search results compared to "love". My hypothesis is that the limited number of documents containing the term "car" — only three in total — provides insufficient data for Latent Semantic Indexing to accurately model related concepts such as `car`, `automobile`, or `vehicle`.

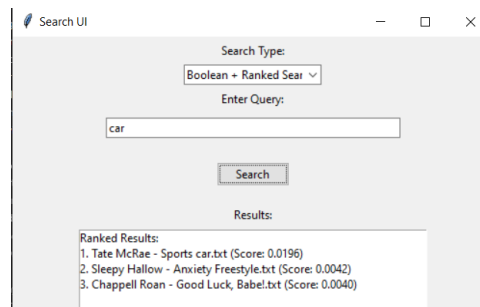


Figure 1: Only 3 (Ranked) results for car

In contrast, for the keyword "love", there were 38 ranked results available, giving LSI a much richer context to identify and group semantically related documents. As a result, LSI was able to capture the underlying concept of romance and emotional themes more effectively.

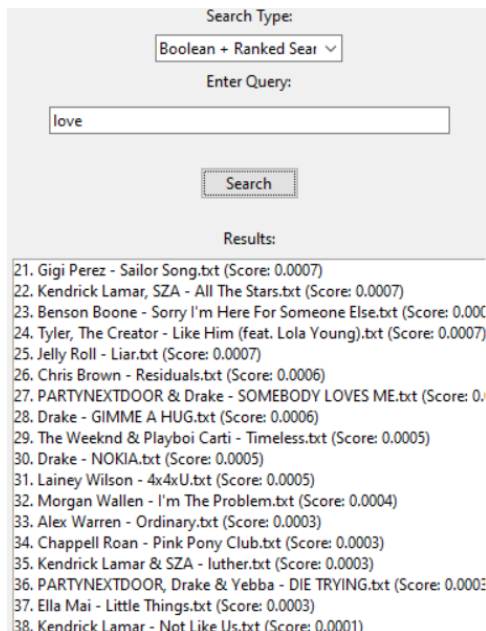


Figure 2: 38 (Ranked) results for love

When using lower values of `n.components` (below 100) for the query "car", the model retrieved documents that had no real connection to the concept of automobiles.

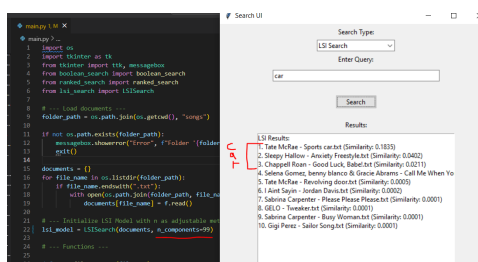


Figure 3: 3 relevant/ 7 irrelevant Documents at $n=99$

This leads to an important conclusion: for LSI to perform reliably, a sufficiently large and thematically diverse dataset is essential. If the dataset is too sparse with respect to certain keywords or topics, LSI will struggle at lower dimensionalities and may associate unrelated documents incorrectly. A strong and well-distributed corpus is therefore a key requirement for effective semantic search.

6.1 Observations

- LSI was able to generalize the meaning of queries, returning relevant documents even when the exact keyword was missing.

- When `n_components` was set too low (e.g., $n = 50$), many unrelated documents appeared, reducing search precision.
- The best balance between semantic understanding and precision was achieved with `n_components` values around 75 to 100.
- Sparse keywords (such as "car") performed worse due to the lack of sufficient related documents in the dataset.

7 Conclusion

The integration of Latent Semantic Indexing (LSI) into the search engine resulted in a noticeable improvement in the relevance and depth of search results. By moving beyond simple keyword matching, the system became capable of identifying semantically related documents, offering a much richer search experience.

Through experimentation, it became evident that:

- Selecting an appropriate number of dimensions (`n_components`) is crucial for balancing detail and generalization.
- LSI performs best with a dataset that offers a broad and deep variety of examples for different topics.
- Sparse datasets or rare keywords can lead to poor semantic modeling, especially when using lower dimensionalities.

While LSI introduced a slight computational overhead due to the need for dimensionality reduction and similarity computations, the gains in search quality outweighed these costs. Overall, LSI proved to be a valuable addition to the search system, particularly for enhancing the retrieval of conceptually related documents in larger, well-distributed corpora. Wich means I will probably have to add more diverse Songs to my corpora.

8 Purpose and Broader Context of This Information Retrieval (IR) System

The general purpose of this Information Retrieval (IR) system, built on a dataset of song lyrics, is to enable users to search for songs not only based on exact keyword matches, but also based on broader semantic associations and partial memories.

A few practical use cases include:

- **Searching by theme or vibe:** Sometimes, users are looking for songs that match a specific mood (e.g., love, sadness, happiness) rather than remembering specific words. An LSI-based system helps retrieve songs that fit the general "vibe" the user is looking for.
- **Partial lyric memory:** Users may only remember a fragment of the lyrics, or they might recall a single word incorrectly. Traditional exact search systems would fail here, but a semantic search could still retrieve the correct or related songs.

- **Discovering related content:** Even when users search for known keywords like "love" or "lovely," the system can suggest related songs that explore similar emotional or thematic content, improving discovery and recommendation.
- **Applications similar to real-world services:** Platforms like Spotify, Apple Music, or Genius rely heavily on advanced IR techniques to suggest songs that fit user queries even when their input is vague, imprecise, or typo-prone. My project models a simplified version of such behavior by demonstrating how latent semantic analysis improves search results for song datasets.

In essence, the goal is to bridge the gap between human memory — which is often fuzzy, emotional, and imprecise — and rigid text-based databases, offering a more intuitive way to find and rediscover music.

References

- [1] Christopher Manning and Pandu Nayak, *Introduction to Information Retrieval*, Ressources from: Scott Deerwester, Susan Dumais, George Furnas, Thomas Landauer, Richard Harshman. 1990. Indexing by latent semantic analysis. JASIS Vol. 41, No. 6, pp. 391–407.
<https://web.stanford.edu/class/cs276/handouts/lecture13-lsi-handout-1-per.pdf>
- [2] Elyesa Duru GitHub, *GitHub-Repository* <https://github.com/freely1967/NLP-and-IR>